

王道考研——组成原理

WWW.CSKAOYAN.COM

第五章 中央处理器

考点17

CPU的功能 和基本结构

考点17: CPU的功能和基本结构

	• 小题18	• 综合题43				• 综合题43 • 综合题44	• 小题18 • 小题20	
2009	2010	2011	2012	2013	2014	2015	2016	2017
• 小题19			• 小题17 • 综合题43	• 综合题43			• 小题18	
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频: 小题×5、综合题×5

CPU的功能和基本结构

【考点笔记】CPU 的基本结构

CPU 主要由运算器与控制器两大部分组成：

- 1) 运算器：执行算术运算、逻辑运算，对数据进行加工和处理；
 - 2) 控制器：执行指令，指令的执行是由控制器发出的一组微操作实现的。
- 运算器与控制器的组成结构如下所示。

运算器	控制器
算术逻辑单元（ALU）	程序计数器（PC）
暂存寄存器（DR）	指令寄存器（IR）
累加寄存器（ACC）	指令译码器
通用寄存器组	存储器地址寄存器（MAR）
程序状态字寄存器（PSWR）	存储器数据寄存器（MDR）
移位器	时序系统
计数器	微操作信号发生器

CPU的功能和基本结构

分类	寄存器	功能
用户可见	通用寄存器组	存放操作数和地址信息；作为基址寄存器、变址寄存器等
	程序状态字寄存器（PSWR）	保留由逻辑运算指令或测试指令的结果建立的各种状态信息
	程序计数器（PC）	指出下一条指令在主存中的存放地址
	累加寄存器（ACC）	暂时存放 ALU 运算的结果信息，用于实现加法运算
用户不可见	指令寄存器（IR）	保存当前正在执行的那条指令
	暂存寄存器（DR）	暂存从主存读来的数据
	存储器地址寄存器（MAR）	存放所要访问的主存单元的地址
	存储器数据寄存器（MDR）	存放向主存写入的信息或从主存中读出的信息

CPU的功能和基本结构



2016_18. 某计算机主存空间为4GB，字长为32位，按字节编址，采用32位字长指令字格式。若指令按字边界对齐存放，则程序计数器（PC）和指令寄存器（IR）的位数至少分别是_____。

- A. 30、30 B. 30、32 C. 32、30 D. 32、32

答案：B

解析：

程序计数器（PC）给出下一条指令字的访存地址（指令在内存中的地址），取决于存储器的字数（ $4\text{GB}/32\text{bit}=2^{30}$ ），故程序计数器（PC）的位数至少是30位；指令寄存器（IR）用于接收取得的指令，取决于指令字长（32位），故指令寄存器（IR）的位数至少为32位。

考点18

指令执行的过程

考点18：指令执行过程



历年考频： 小题×2、综合题×3

指令执行过程

【考点笔记】指令周期

指令周期是指从取指令、分析取数到执行完该指令所需的全部时间。指令周期通常包含若干个机器周期，每个机器周期完成一个基本操作。一个机器周期又包含若干时钟周期（也称节拍，它是 CPU 操作的最基本单位）。一个完整的指令周期如图 5-1 所示。

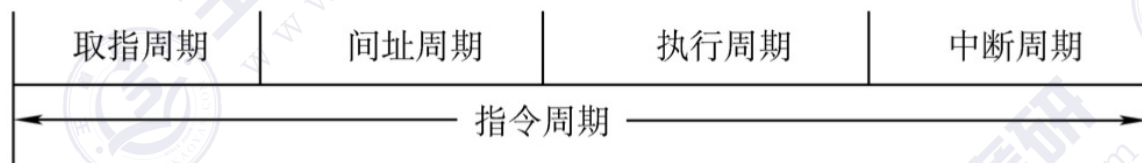


图 5-1 带有间址周期、中断周期的指令周期

上述 4 个周期都可能有 CPU 访存，只是目的不同。取指周期是为了取指令，间址周期是为了取有效地址，执行周期是为了取操作数，中断周期是为了保存程序断点。

指令执行过程

机器周期	数据流向	标志位
取址周期	① PC→MAR→地址总线→主存 ② CU 发出读命令→控制总线→主存 ③ 主存→数据总线→MDR→IR（存放指令） ④ CU 发出读命令→PC 内容加 1	1→FE
间址周期	① Ad(IR)（或 MDR）→MAR→地址总线→主存 ② CU 发出读命令→控制总线→主存 ③ 主存→数据总线→MDR（存放有效地址）	1→IND
执行周期	不同指令的执行周期操作不同	1→EX
中断周期	① CU 控制将 SP 减 1，SP→MAR→地址总线→主存 ② CU 发出写命令→控制总线→主存 ③ PC→MDR→数据总线→主存（程序断点存入主存） ④ CU（中断服务程序的入口地址）→PC	1→INT

指令执行过程

2011_19. 假定不采用Cache和指令预取技术，且机器处于“开中断”状态，则在下列有关指令执行的叙述中，错误的是_____。

- A. 每个指令周期中CPU都至少访问内存一次
- B. 每个指令周期一定大于或等于一个CPU时钟周期
- C. 空操作指令的指令周期中任何寄存器的内容都不会被改变
- D. 当前程序在每条指令执行结束时都可能被外部中断打断

答案：C

解析：

由于不采用指令预取技术，每个指令周期都需要取指令，而不采用Cache技术，则每次取指令都至少要访问内存一次（当指令字长与存储字长相等且按边界对齐时），A正确。时钟周期是CPU的最小时间单位，每个指令周期一定大于或等于一个CPU时钟周期，B正确。即使是空操作指令，在取指操作后，PC也会自动加1，C错误。由于机器处于“开中断”状态，在每条指令执行结束时都可能被外部中断打断。

考点19

数据通路的功能和基本结构

考点19：数据通路的功能和基本结构

• 综合题44 2009	• 综合题43 2010			• 综合题44 2013		• 综合题43 2015	• 小题20 2016	• 小题19 2017
				• 小题18 2021	• 综合题43 2022	• 小题18 2023	• 大题43 2024	• 小题19 2025

历年考频： 小题×5、综合题×6

数据通路的功能和基本结构

【考点笔记】数据通路

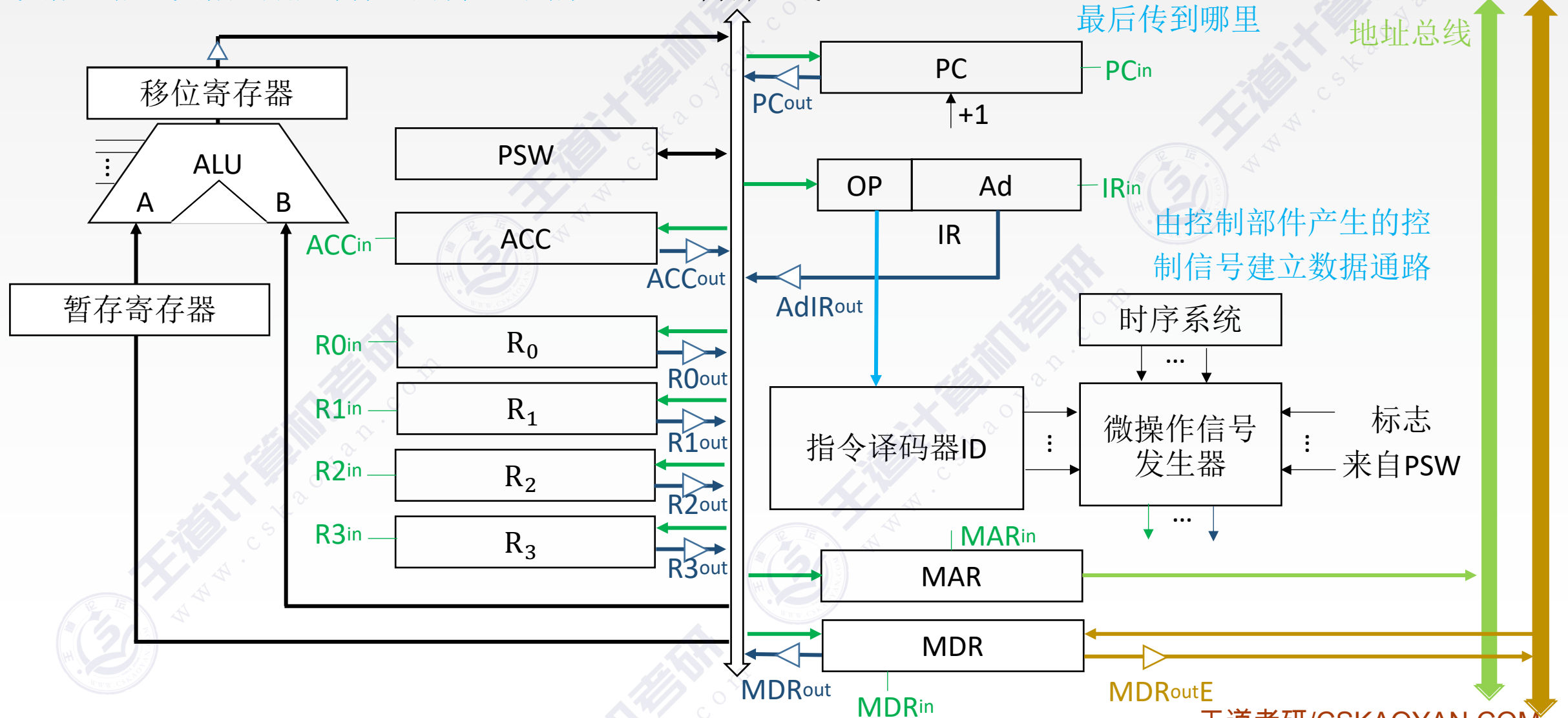
数据通路是 CPU 中逻辑运算单元（ALU）、控制单元（CU）及寄存器之间的连接线路。

控制器在实现一条指令的功能时，总要把每条指令分解成一系列时间上先后有序的最基本、最简单的微操作，即微操作序列。微操作序列是与 CPU 的内部数据通路密切相关的，不同的数据通路就有不同的微操作序列。

一般在给出 CPU 的结构及数据通路框图之后，再写出对应指令的微操作序列。如果 CPU 内部采用单总线结构，还要考虑总线冲突的问题，相应的微操作控制信号必须与给出的数据通路结构一致，且时序上要有先后顺序。

数据通路(单总线结构)

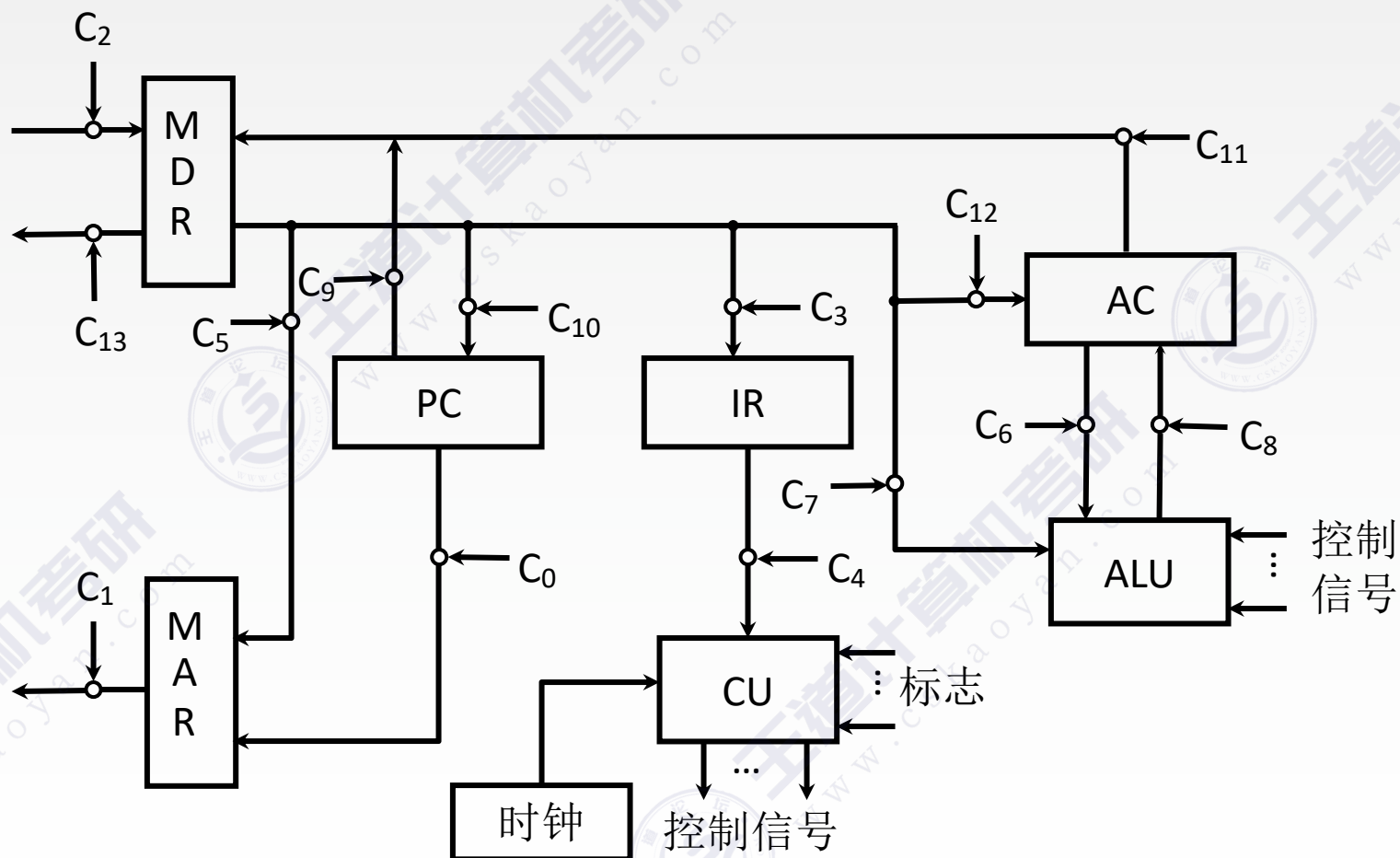
数据通路：数据在功能部件之间传送的路径。 CPU内部总线



信息从哪里开始
中间经过哪些部件
最后传到哪里

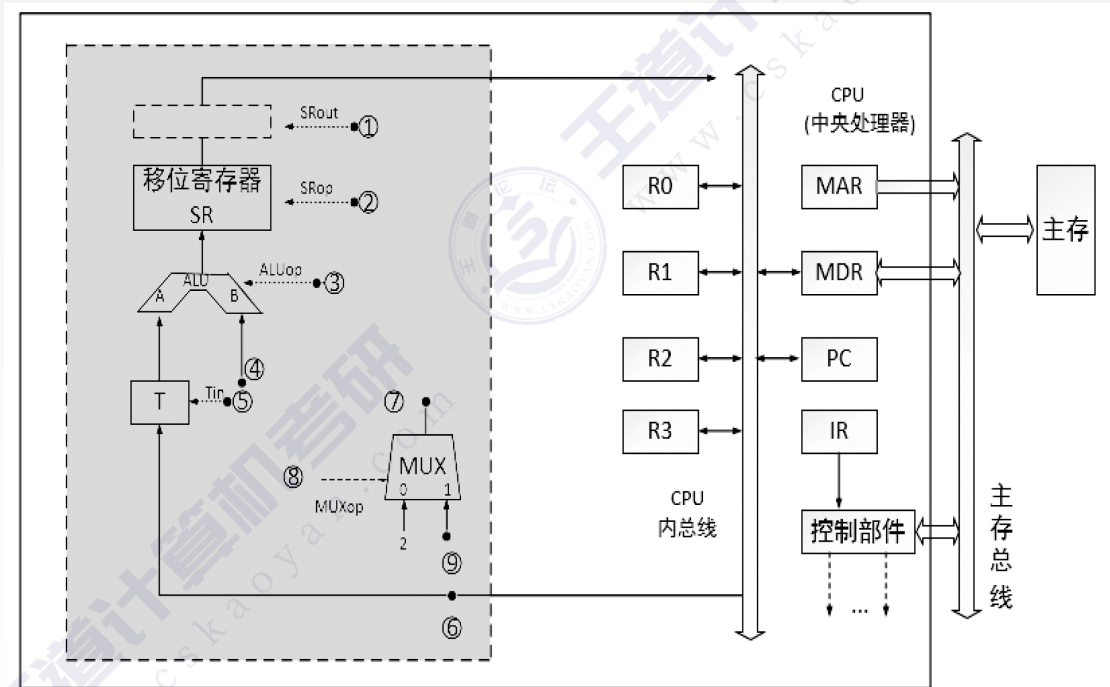
由控制部件产生的控制
信号建立数据通路

专用数据通路方式



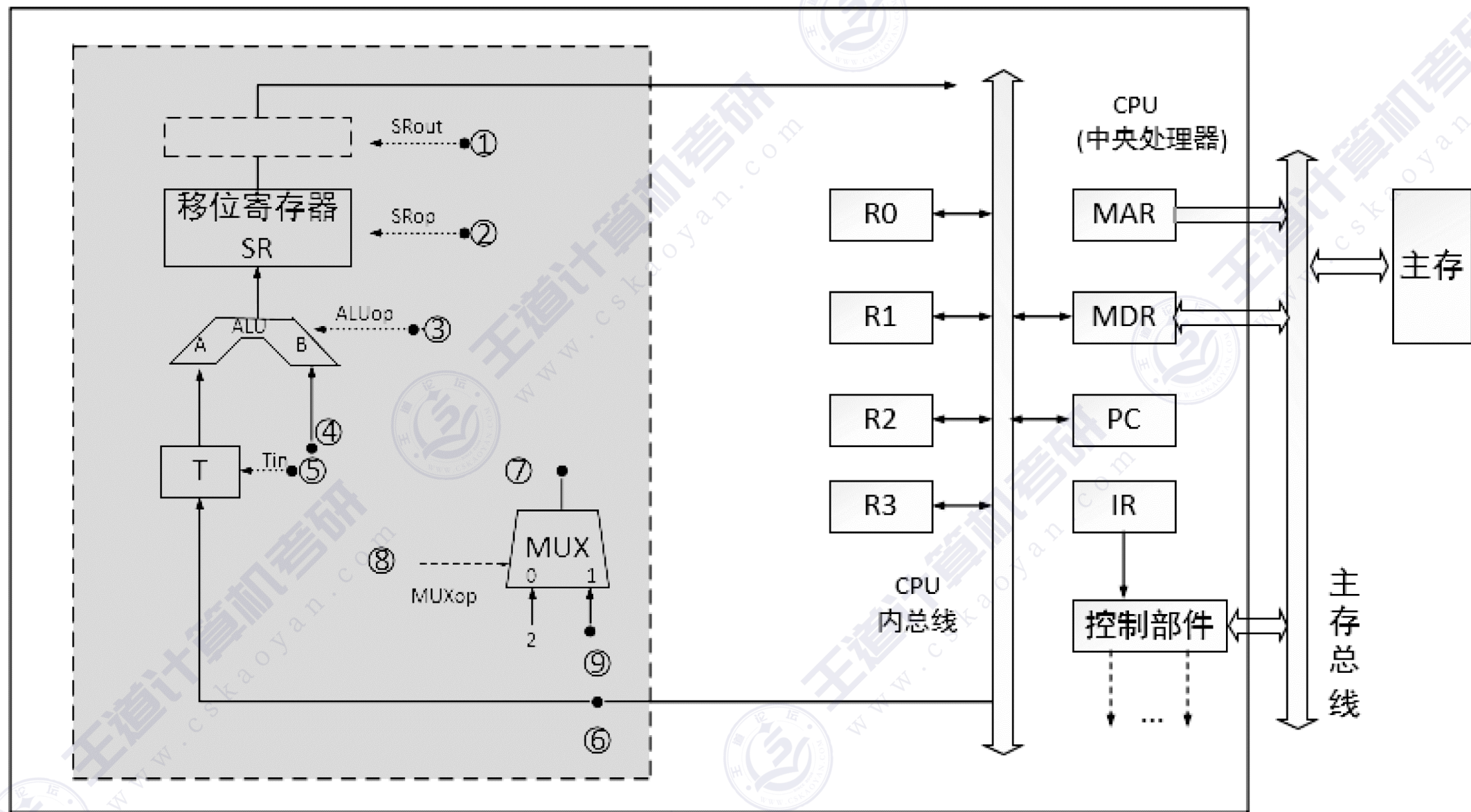
单周期处理器常采用“专用数据通路”

2015_43. 某16位计算机的主存按字节编码，存取单位为16位；采用16位定长指令字格式；CPU采用单总线结构，主要部分如下图所示。图中R0~R3为通用寄存器；T为暂寄存器；SR为移位寄存器，可实现直送（mov）、左移一位（left）和右移一位（right）3种操作，控制信号为SRop，SR的输出由信号SRout控制；ALU可实现直送A（mova）、A加B（add）、A减B（sub）、A与B（and）、A或B（or）、非A（not）、A加1（inc）7种操作，控制信号为ALUop。



请回答下列问题。

- 1) 图中哪些寄存器是程序员可见的？为何要设置暂寄存器T？
- 2) 控制信号ALUop和SRop的位数至少各是多少？
- 3) 控制信号SRout所控制部件的名称或作用是什么？
- 4) 端点①~⑨中，哪些端点须连接到控制部件的输出端？
- 5) 为完善单总线数据通路，需要在端点①~⑨中相应的端点之间添加必要的连线。写出连线的起点和终点，以正确表示数据的流动方向。
- 6) 为什么二路选择器MUX的一个输入端是2？



考点20

控制器的功能 和工作原理

考点20：控制器的功能和工作原理

• 小题19 • 综合题44 2009 	2010 	• 小题19 2011 	• 小题18 2012 	2013 	• 小题18 • 综合题45 2014 	• 综合题43 • 综合题44 2015 	2016 	• 小题18 • 综合题44 2017 
2018 	• 小题16 2019 	2020 	2021 	• 综合题43 2022 	2023 	2024 	2025 	

历年考频： 小题×6、 综合题×6

控制器的功能和基本原理

【考点笔记】控制器的结构与分类

硬布线控制器和微程序控制器的设计思想和特点见下表。

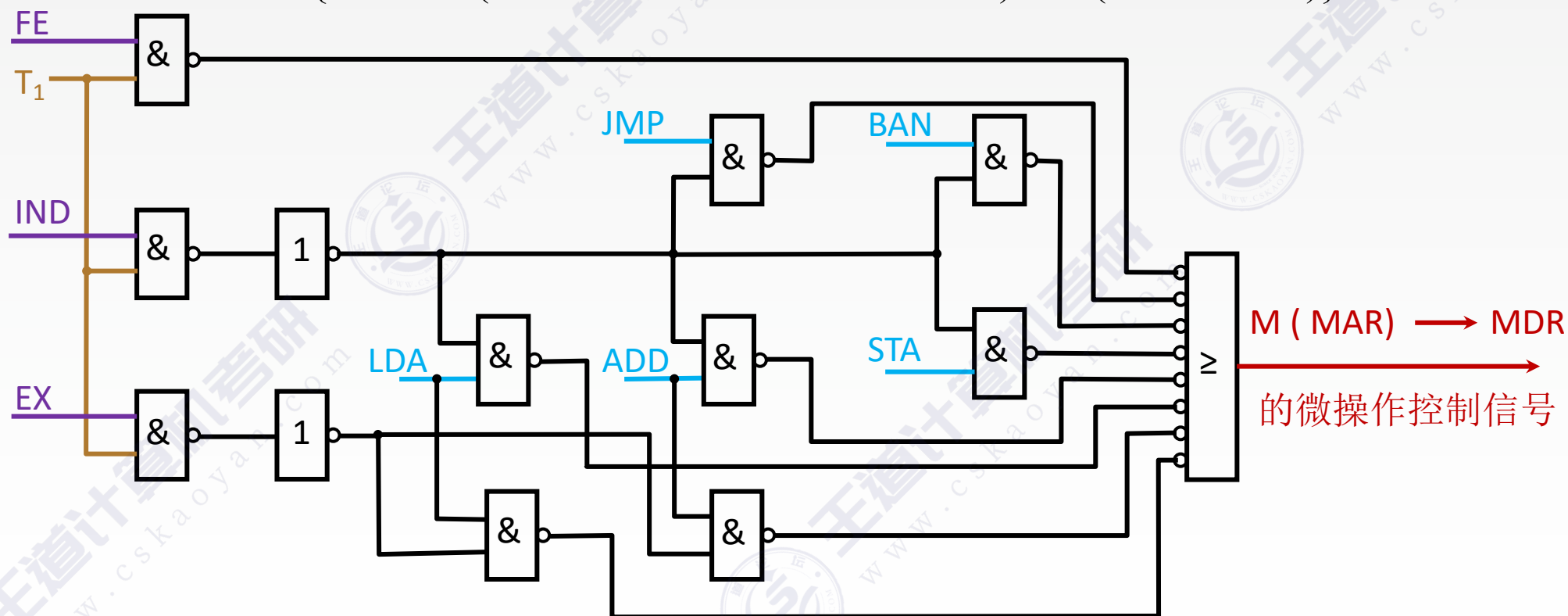
控制方式	设计思想	特点
硬布线控制器	微操作控制信号由组合逻辑电路（由门电路与触发器构成的复杂网络）根据当前的指令码、状态和时序，即时产生	控制器的速度取决于电路延迟，速度快 逻辑线路固定，难以扩充和修改 结构复杂，不规整 通常应用于 RISC 结构的 CPU
微程序控制器	采用存储程序的原理，将微操作控制信号以微程序的形式存放在控制存储器中，一条机器指令对应一个微程序，执行指令时读出微程序执行即可	每条指令都要从控存中读取，速度较慢 可通过改变控存中的内容进行扩充和修改 结构规整 通常应用于 CISC 结构的 CPU

硬布线控制器

M (MAR) → MDR 微操作命令的逻辑表达式:

$$FE \cdot T_1 + IND \cdot T_1 (ADD + STA + LDA + JMP + BAN) + EX \cdot T_1 (ADD + LDA)$$

$$= T_1 \{ FE + IND (ADD + STA + LDA + JMP + BAN) + EX (ADD + LDA) \}$$



根据 指令操作码、目前的机器周期、节拍信号、机器状态条件，即可确定
现在这个节拍下应该发出哪些“微命令”

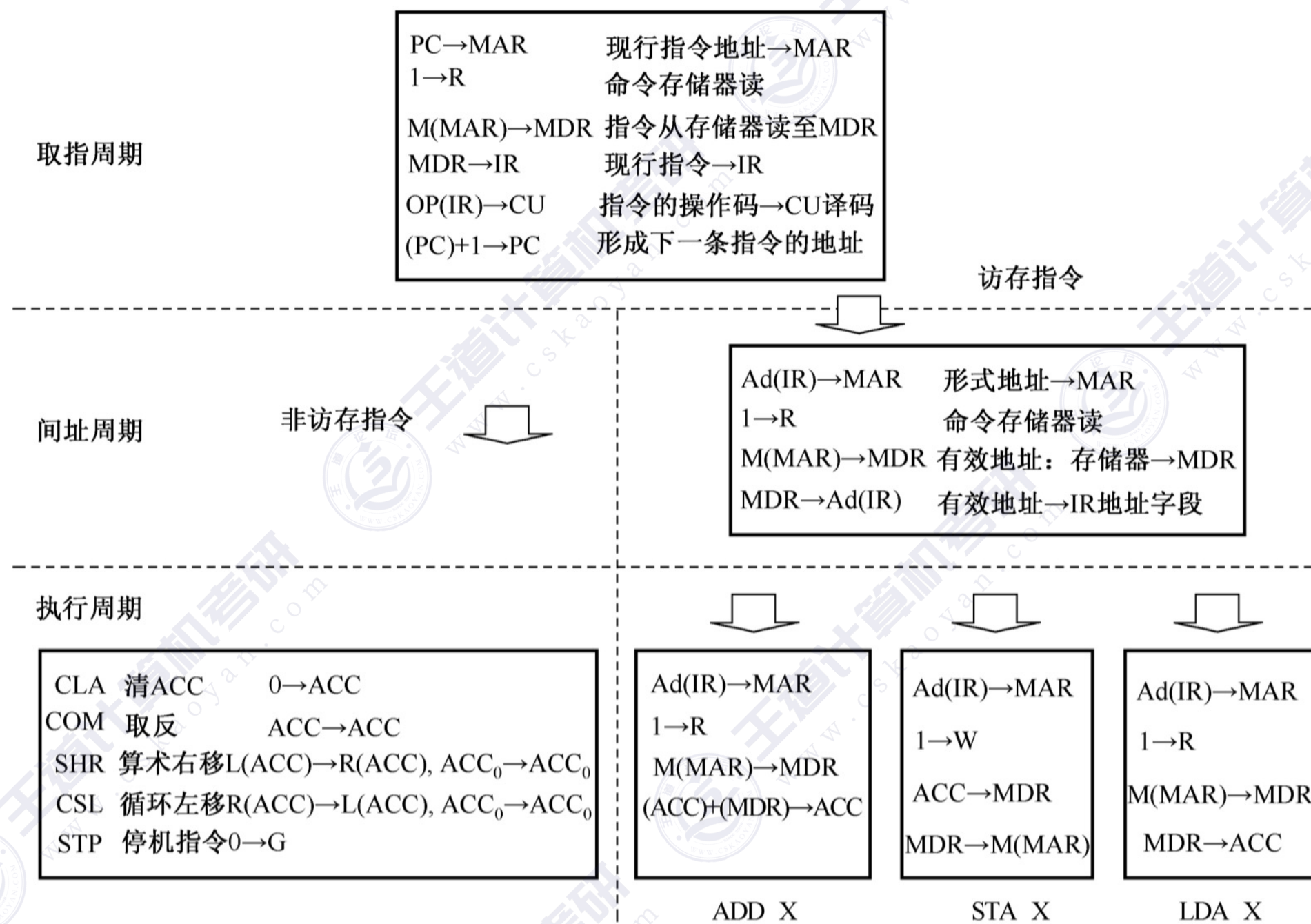


图 5-2 硬布线控制器的微操作指令流程图

控制器的功能和工作原理

2009_19. 相对于微程序控制器，硬布线控制器的特点是_____。

- A. 指令执行速度慢，指令功能的修改和扩展容易
- B. 指令执行速度慢，指令功能的修改和扩展难
- C. 指令执行速度快，指令功能的修改和扩展容易
- D. 指令执行速度快，指令功能的修改和扩展难

答案：D

解析：

微程序控制器采用了“存储程序”的原理，每条机器指令对应一个微程序，因此修改和扩充容易，灵活性好，但每条指令的执行都要访问控制存储器，所以速度慢。硬布线控制器采用专门的逻辑电路实现，其速度主要取决于逻辑电路的延迟，因此速度快，但修改和扩展困难，灵活性差。

微程序控制器的设计思路

采用“存储程序”的思想，CPU 出厂前将所有指令的“微程序”存入“控制器存储器”中



程序：由指令序列组成

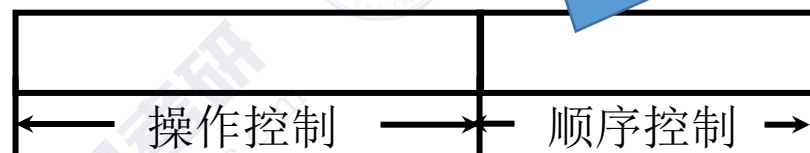
微程序：由微指令序列组成，每一种指令对应一个微程序

指令是对程序执行步骤的描述

微指令是对指令执行步骤的描述

指令是对微指令功能的“封装”

微指令基本格式

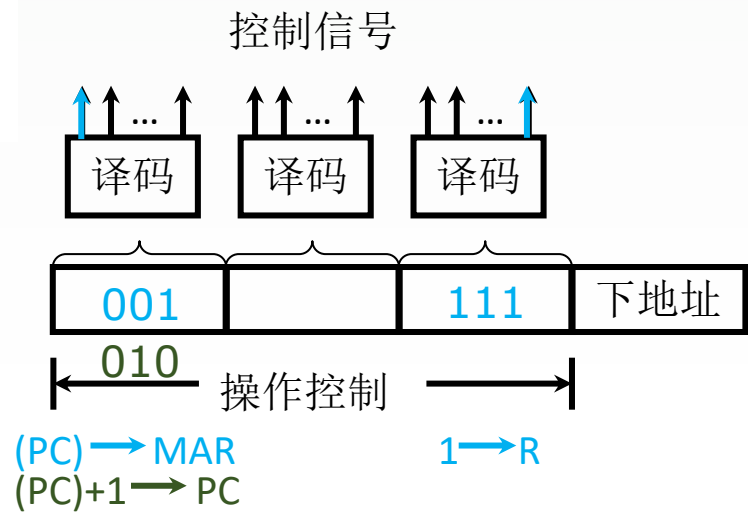
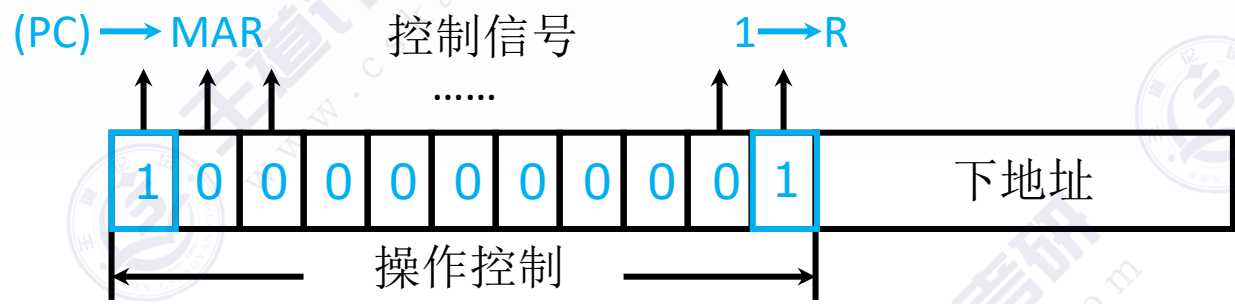


微命令与微操作一一对应
微指令中可能包含多个微命令

【考点笔记】微指令的编码方式

两种主要编码方式的对比见下表。

编码方式	直接编码	字段直接编码
实现方法	微指令的控制字段中每一位都代表一个微命令。设计微指令时,选用或不选用某个微命令,只要将表示该微命令的对应位设置成 1 或 0 即可	将微指令的微命令字段分成若干小字段,把互斥性微命令组合在同一字段中,把相容性微命令组合在不同的字段中,每个字段独立编码,每种编码代表一个微命令且各字段编码含义单独定义,与其他字段无关
特点	简单、直观;指令字长过长, n 个微命令就要求微指令的操作字段有 n 位	微命令字段分段的原则: 1) 互斥性微命令分在同一段内,相容性微命令分在不同段内 2) 每个小段中包含的信息位不能太多,否则将增加译码线路的复杂性和译码时间 3) 一般每个小段还要留出一个状态,表示本字段不发出任何微命令



控制器的功能和工作原理

2012_18. 某计算机的控制器采用微程序控制方式，微指令中的操作控制字段采用字段直接编码法，共有33个微命令，构成5个互斥类，分别包含7、3、12、5和6个微命令，则操作控制字段至少有_____。

- A. 5位 B. 6位 C. 15位 D. 33位

答案：C

解析：

字段直接编码法将微命令字段分成若干个小字段，互斥性微命令组合在同一字段中，相容性微命令分在不同字段中，每个字段还要留出一个状态，表示本字段不发出任何微命令。5个互斥类，分别包含7、3、12、5和6个微命令，需要3、2、4、3和3位，共15位。

控制器的功能和基本原理

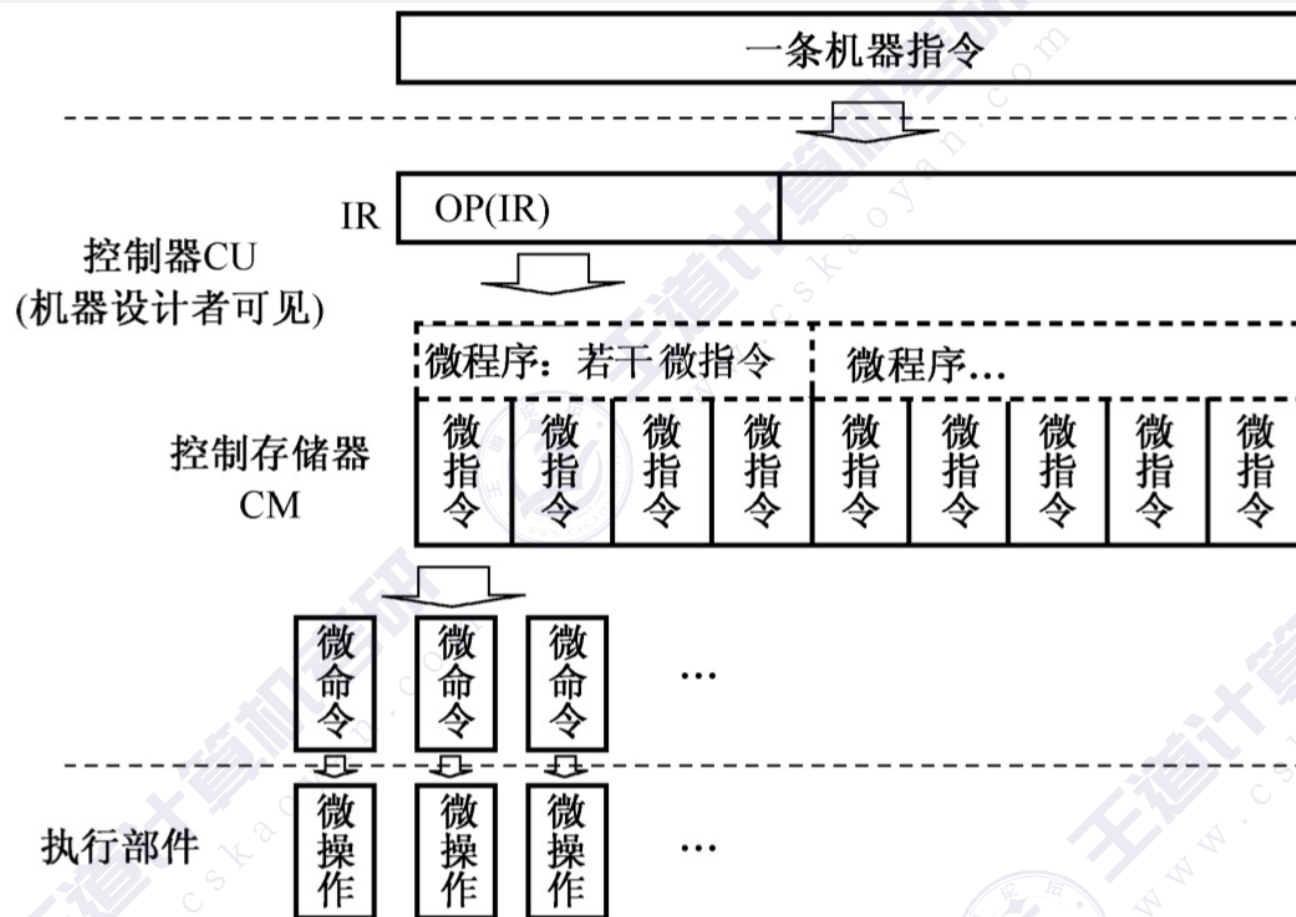


图 5-3 微硬程序控制器的工作流程图

考点21

指令流水线

考点21：指令流水线

• 小题18	• 小题19	• 小题18	• 综合题44	• 小题18	• 小题16 • 综合题44		• 小题19	• 小题17
2009	2010	2011	2012	2013	2014	2015	2016	2017
• 小题20	• 小题18	• 小题17			• 小题19	• 小题19		
2018	2019	2020	2021	2022	2023	2024	2025	

历年考频： 小题×12、综合题×2

指令流水线

【考点笔记】指令流水线

将一条指令的执行过程分成多个子过程，每个子过程与其他子过程通过各自独立的功能部件并行执行。图 5-4 所示为四个功能段的指令流水线的时空图。

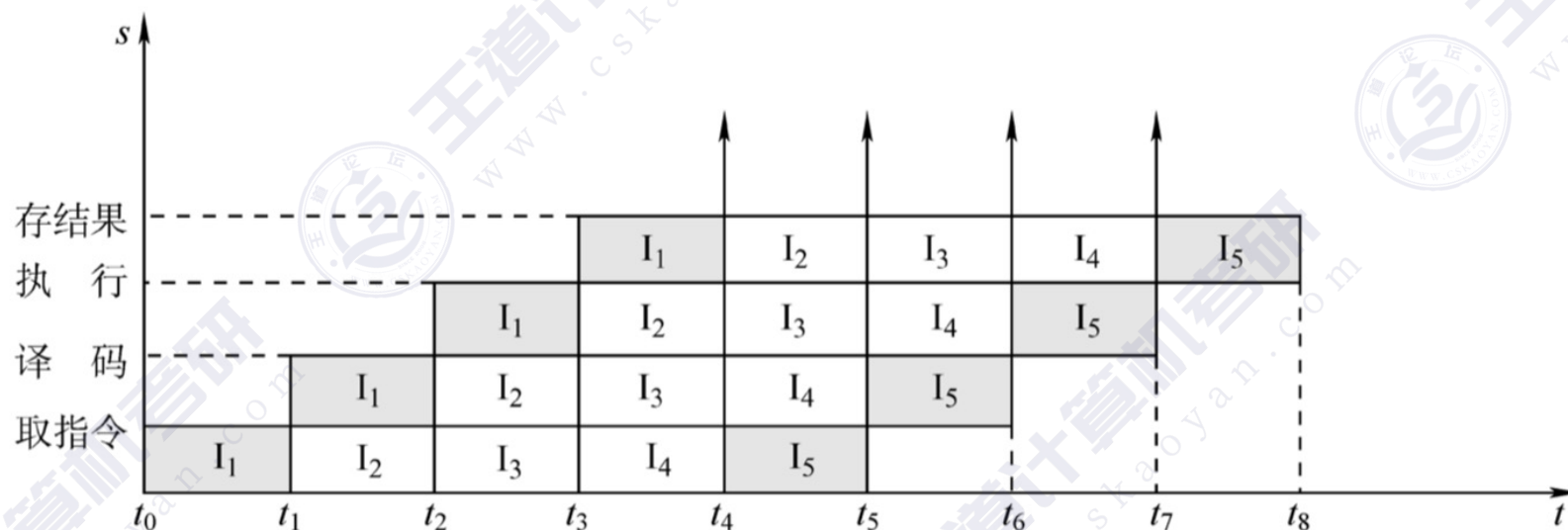


图 5-4 四个功能段的指令流水线时空图

流水线的每个子过程由专用的功能段实现，各功能段所需时间应尽量相等。否则，时间长的功能段将成为流水线的瓶颈。

指令流水线



2018_20. 若某计算机最复杂指令的执行需要完成5个子功能，分别由功能部件A~E实现，各功能部件所需时间分别为80ps、50ps、50ps、70ps和50ps，采用流水线方式执行指令，流水段寄存器延时为20ps，则 CPU时钟周期至少为_____。

A. 60 ps

B. 70 ps

C. 80 ps

D. 100 ps

答案：D

解析：

指令流水线的每个流水段时间单位为时钟周期，题中指令流水线的指令需要用到A-E，5个部件，所以每个流水段时间应取：最大部件时间80ps，此外还有寄存器延时为20ps，则CPU时钟周期至少是100ps。答案选D。

指令流水线阻塞的因素

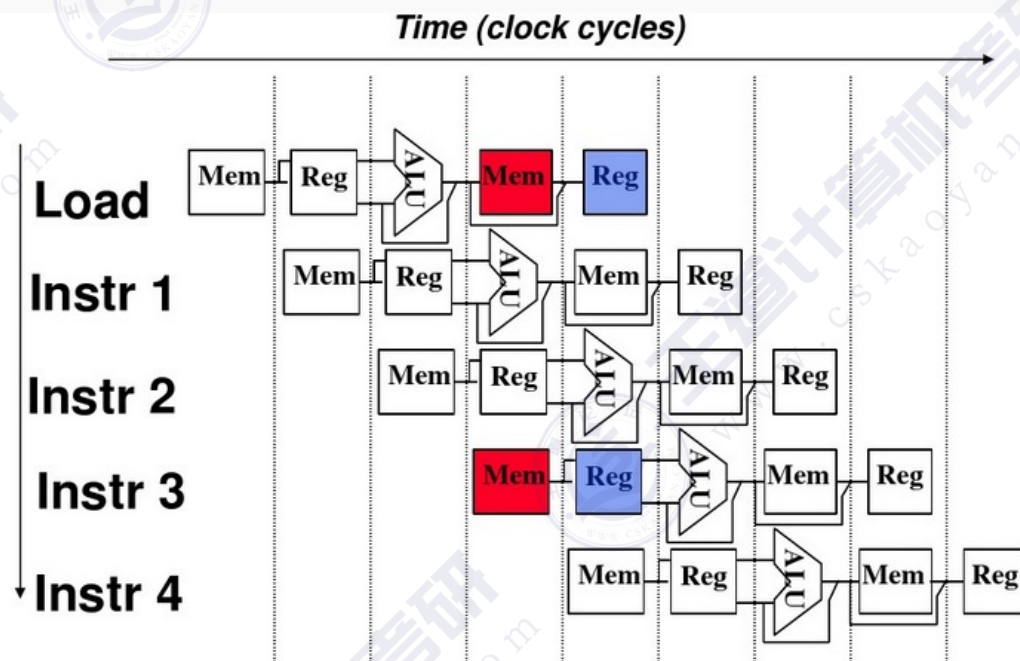
【考点笔记】流水线阻塞的因素

(1) 结构相关

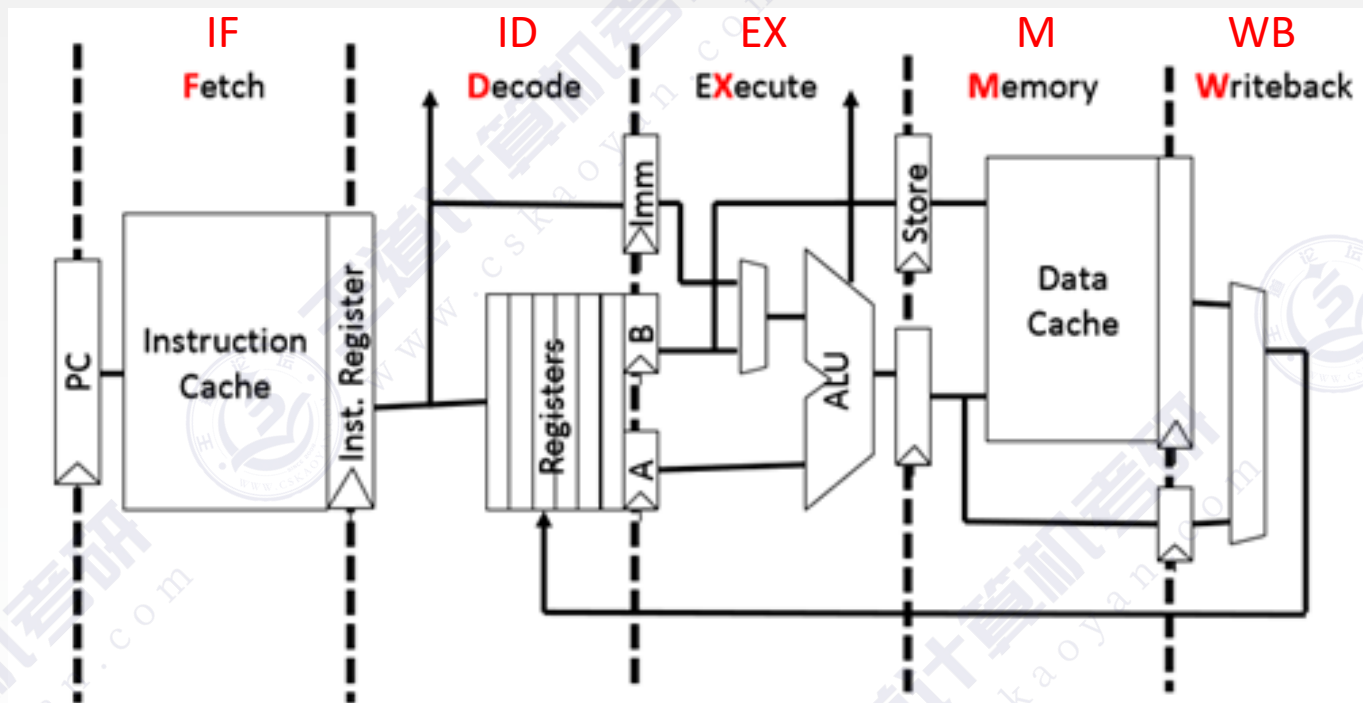
由于多条指令在同一时刻争用同一资源而形成的冲突称为结构相关。

解决办法有以下两种：

- 1) 前一指令访存时，使后一条相关指令（以及其后续指令）暂停一个时钟周期。
- 2) 单独设置数据存储器 and 指令存储器，使两项操作各自在不同的存储器中进行，这属于资源重复配置。



五段式指令流水线



流水线每一个功能段部件后面都要有一个缓冲寄存器，或称为锁存器，其作用是保存本流水段的执行结果，提供给下一流水段使用。

理想情况下，每个机器周期（功能段）只消耗一个时钟周期。

指令流水线阻塞的因素

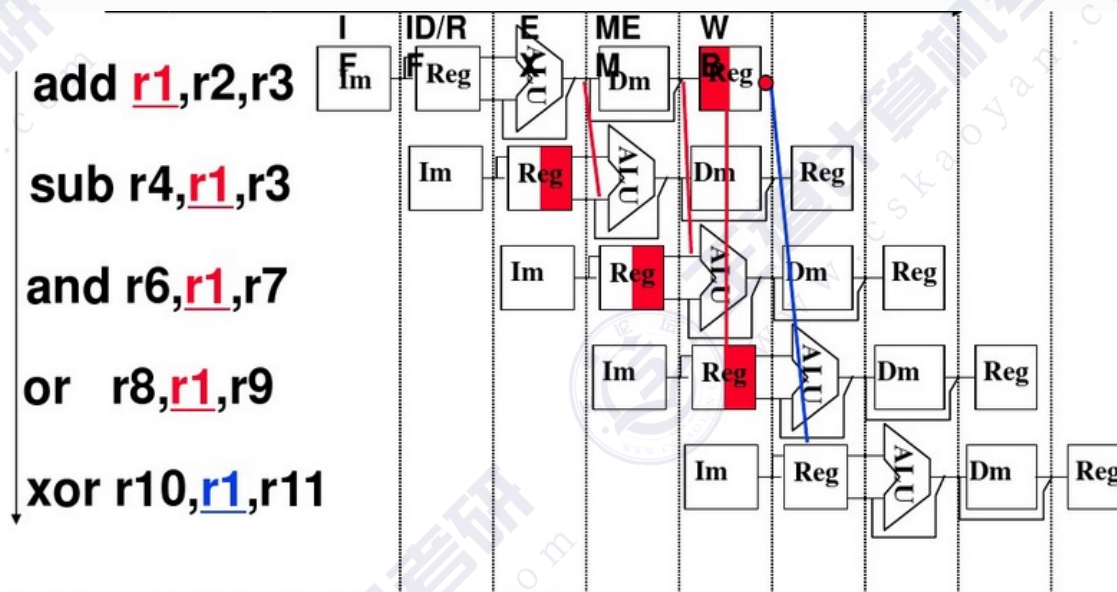
(2) 数据相关

数据相关指在一个程序中，存在必须等前一条指令执行完才能执行后一条指令的情况，那么这两条指令即为数据相关。

解决办法有以下两种：

1) 把遇到数据相关的指令及其后续指令都暂停一至几个时钟周期，直到数据相关问题消失后再继续执行。

2) 设置相关专用通路，即不等前一条指令把计算结果写回寄存器组，下一条指令也不再读寄存器组，而是直接把前一条指令的 ALU 的计算结果作为自己的输入数据开始计算过程，使本来需要暂停的操作可以继续执行，称之为数据旁路技术。



指令流水线

2016_19. 在无转发机制的五段基本流水线（取指、译码/读寄存器、运算、访写回寄存器）中，下列指令序列存在数据冒险的指令对是_____。

I1: add R1, R2, R3; $(R2) + (R3) \rightarrow R1$

I2: add R5, R2, R4; $(R2) + (R4) \rightarrow R5$

I3: add R4, R5, R3; $(R5) + (R3) \rightarrow R4$

I4: add R5, R2, R6; $(R2) + (R6) \rightarrow R5$

A. I1和I2

B. I2和I3

C. I2和I4

D. I3和I4

答案：B

解析：

数据冒险，即数据相关，指在一个程序中存在必须等前一条指令执行完才能执行后一条指令的情况，则这两条指令即为数据相关。当多条指令重叠处理时就会发生冲突。首先这两条指令发生写后读相关，并且两条指令在流水线中执行情况（发生数据冒险）如下图所示：

指令	1	2	3	4	5	6	7
I2	取指	译码/读寄存器	运算	访存	写回		
I3		取指	译码/读寄存器	运算	访存	写回	

指令I2在时钟5时将结果写入寄存器（R5），但指令I3在时钟3时读寄存器（R5）。本来指令I2应先写入R5，指令I3后读R5，结果变成指令I3先读R5，指令I2后写入R5，因而发生数据冲突。

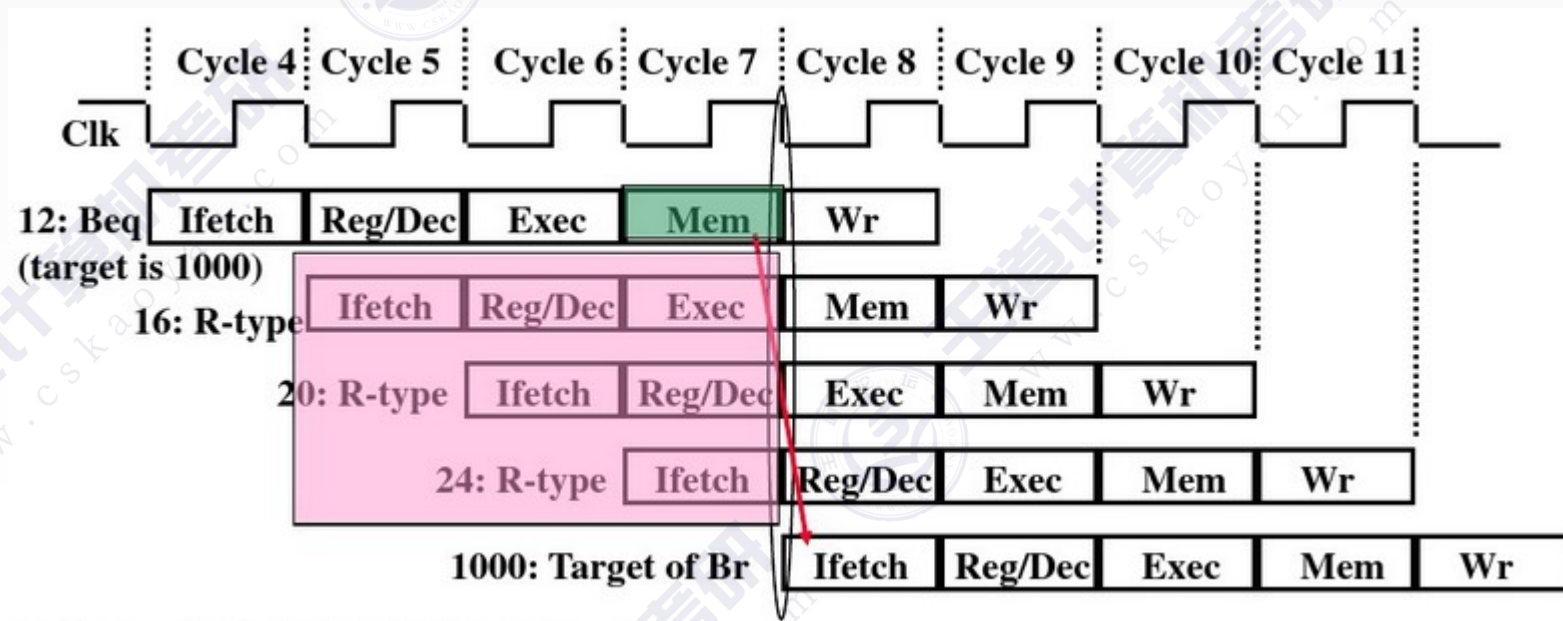
指令流水线阻塞的因素

(3) 控制相关（控制冲突）

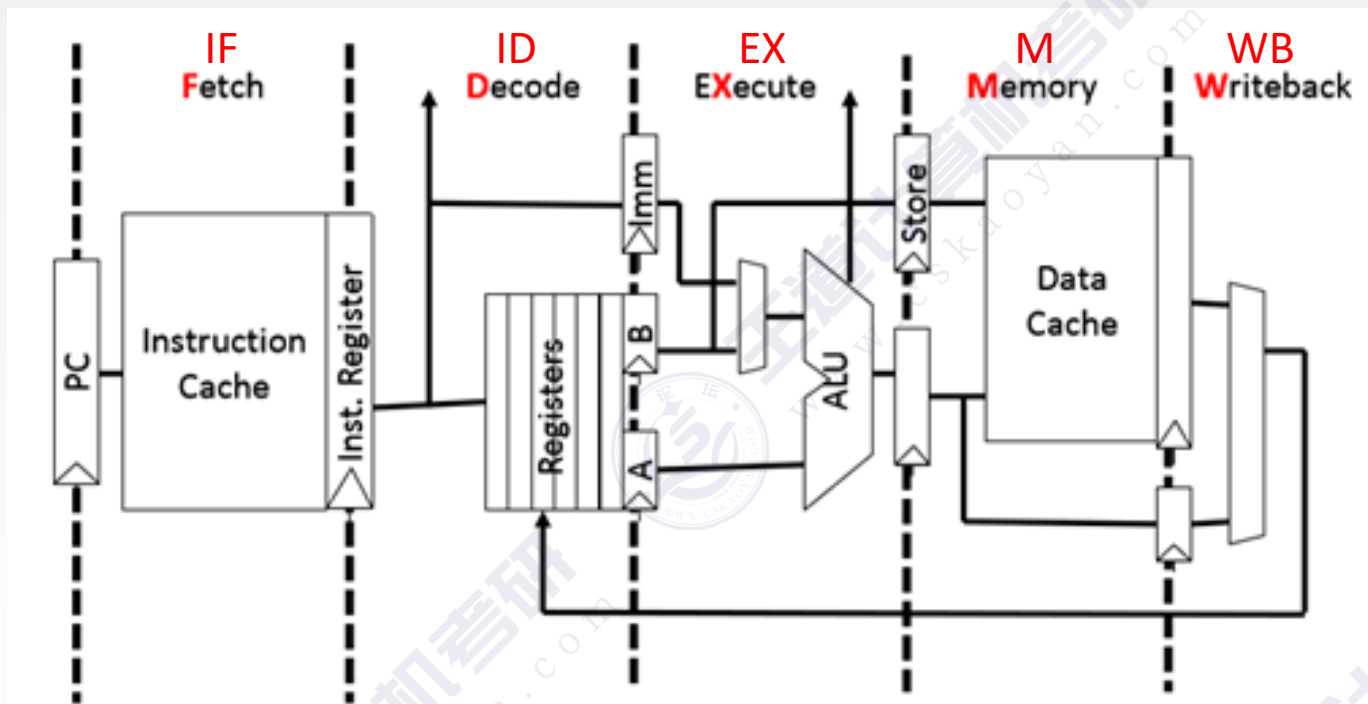
当流水线遇到转移指令和其他改变 PC 值的指令而造成断流时，会引起控制相关。

解决办法有以下几种：

- 1) 尽早判别转移是否发生，尽早生成转移目标地址。
- 2) 预取转移成功和不成功两个控制流方向上的目标指令。
- 3) 加快和提前形成条件码。
- 4) 提高转移方向的猜准率。



条件转移指令的执行过程



转移类指令常采用相对寻址

条件转移指令

IF: 根据PC从指令Cache取指令至IF段的锁存器

ID: 进行比较的两个数放入锁存器A、B；偏移量放入 Imm

EX: 运算，比较两个数

M: 将目标PC值写回PC（左图没画全）

WB: 空段

很多教材把写回PC的功能段称为“WrPC段”，其耗时比M段更短，可安排在M段时间内完成

①IF取指 → ②ID译码&取数 → ③EX 执行 → ④M访存 → ⑤WB写回寄存器

注：通常在IF段结束止之后PC就会自动 + “1”

指令的汇编格式

beq Rs, Rt, #偏移量

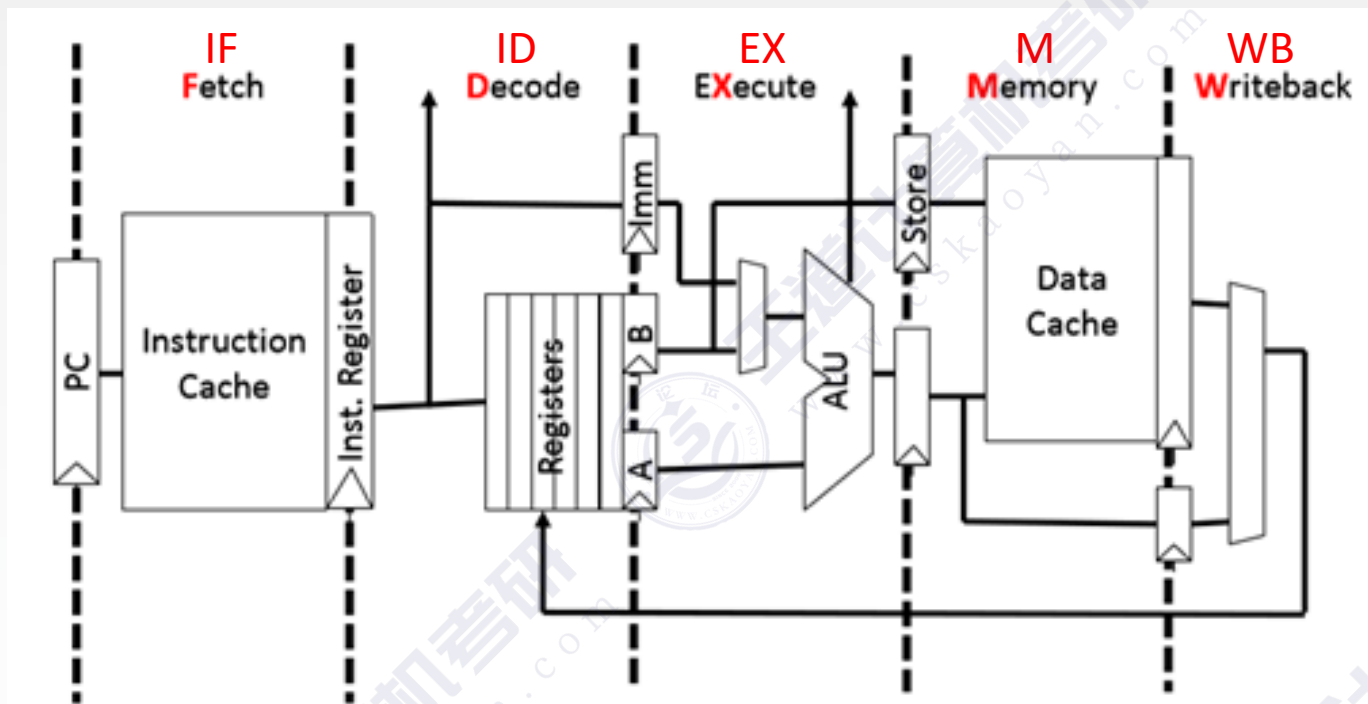
bne Rs, Rt, #偏移量

功能

若(Rs)==(Rt),则(PC)+指令字长+(偏移量×指令字长)→PC； 否则(PC)+指令字长→PC

若(Rs)!=(Rt),则(PC)+指令字长+(偏移量×指令字长)→PC； 否则(PC)+指令字长→PC

运算类指令的执行过程



注:

R_s 指源操作数 (source)

R_d 指目的操作数 (destination)

运算类指令

IF: 根据PC从指令Cache取指令至IF段的锁存器

ID: 取出操作数至ID段锁存器

EX: 运算, 将结果存入EX段锁存器

M: 空段

WB: 将运算结果写回指定寄存器

①IF取指 → ②ID译码&取数 → ③EX 执行 → ④M访存 → ⑤WB写回寄存器

运算类指令举例

加法指令 (两个寄存器相加):

加法指令 (寄存器与立即数相加):

算数左移指令:

指令的汇编格式

ADD R_s, R_d

ADD #996, R_d

SHL R_d

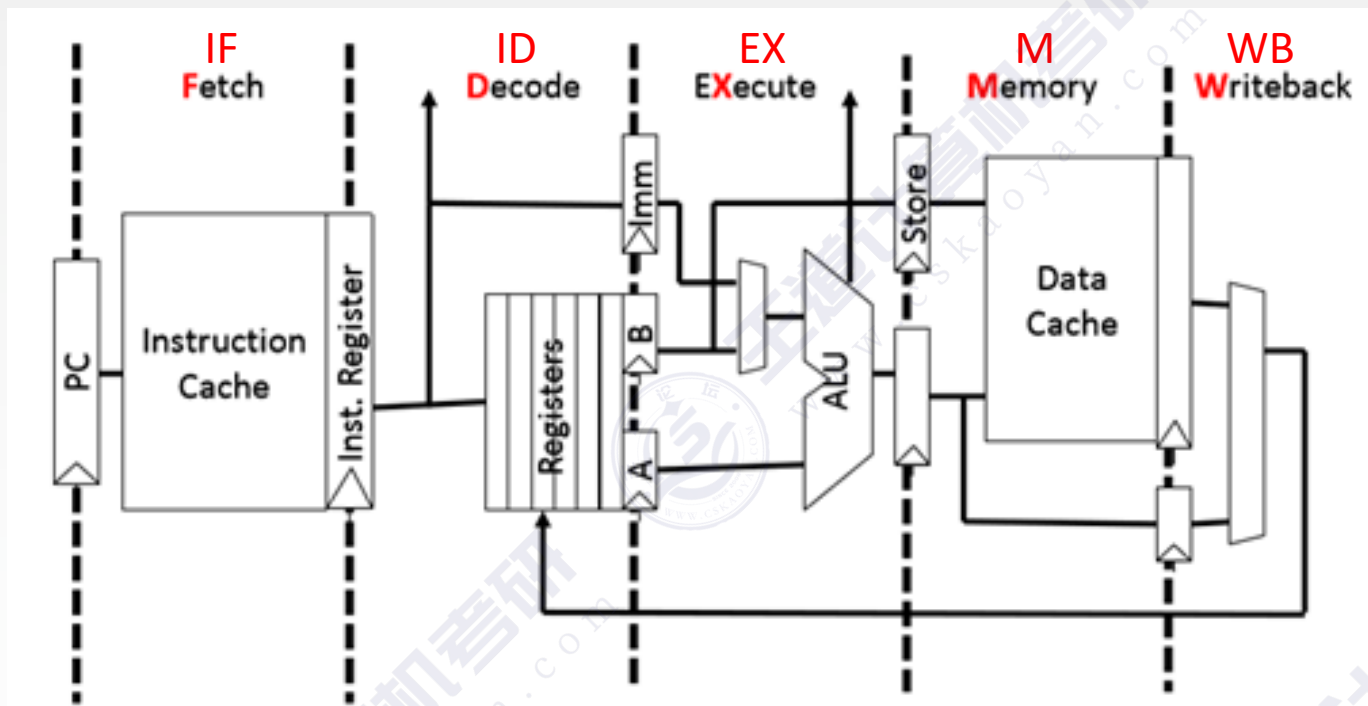
功能

$(R_s) + (R_d) \rightarrow R_d$

$996 + (R_d) \rightarrow R_d$

$(R_d) \ll 2 \rightarrow R_d$

LOAD指令的执行过程



LOAD指令

IF: 根据PC从指令Cache取指令至IF段的锁存器

ID: 将基址寄存器的值放到锁存器A，将偏移量的值放到Imm

EX: 运算，得到有效地址

M: 从数据Cache中取数并放入锁存器

WB: 将取出的数写回寄存器

①IF取指 → ②ID译码&取数 → ③EX 执行 → ④M访存 → ⑤WB写回寄存器

指令的汇编格式

LOAD Rd,996(Rs)

功能

$(996 + (Rs)) \rightarrow Rd$

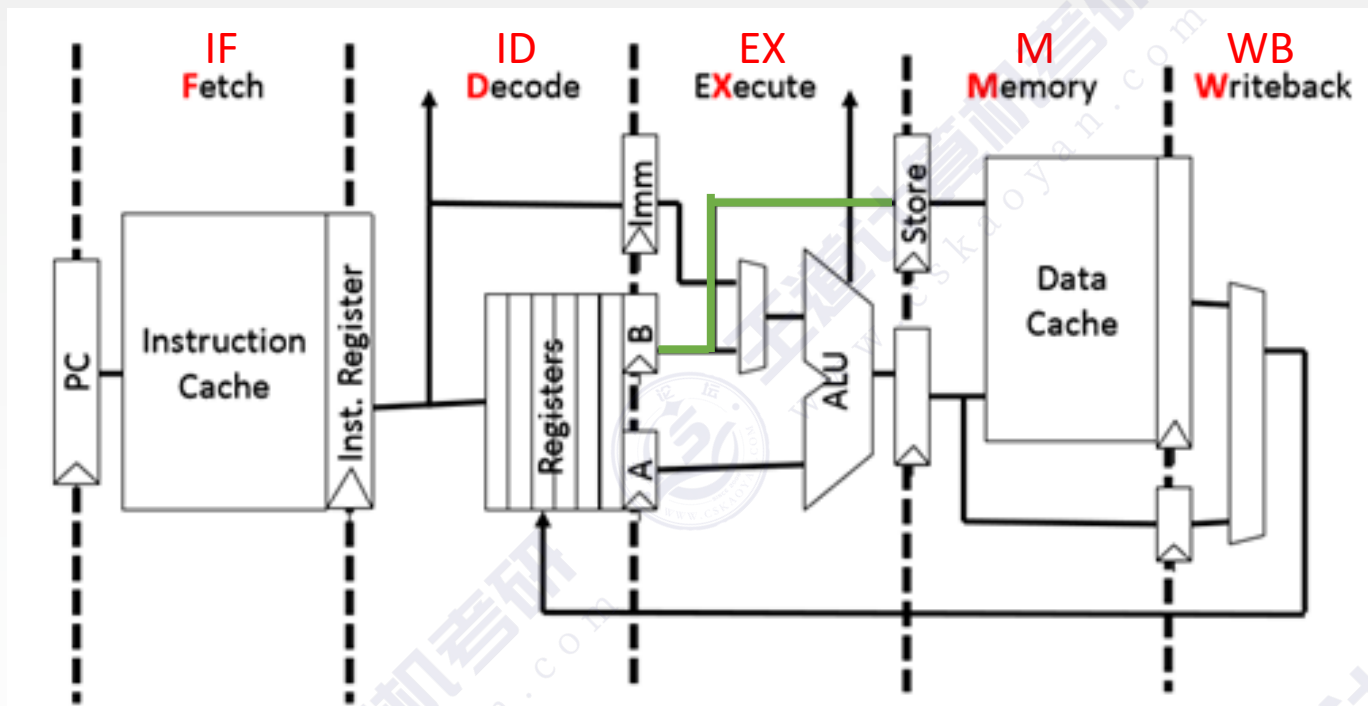
或简写为:

LOAD Rd,mem

$(mem) \rightarrow Rd$

通常，RISC处理器只有“取数LOAD”和“存数STORE”指令才能访问主存

STORE指令的执行过程



STORE指令

IF: 根据PC从指令Cache取指令至IF段的锁存器

ID: 将基址寄存器的值放到锁存器A，将偏移量的值放到Imm。将要存的数放到B

EX: 运算，得到有效地址。并将锁存器B的内容放到锁存器 Store。

M: 写入数据Cache

WB: 空段

①IF取指 → ②ID译码&取数 → ③EX 执行 → ④M访存 → ⑤WB写回寄存器

指令的汇编格式

STORE Rs,996(Rd)

或简写为:

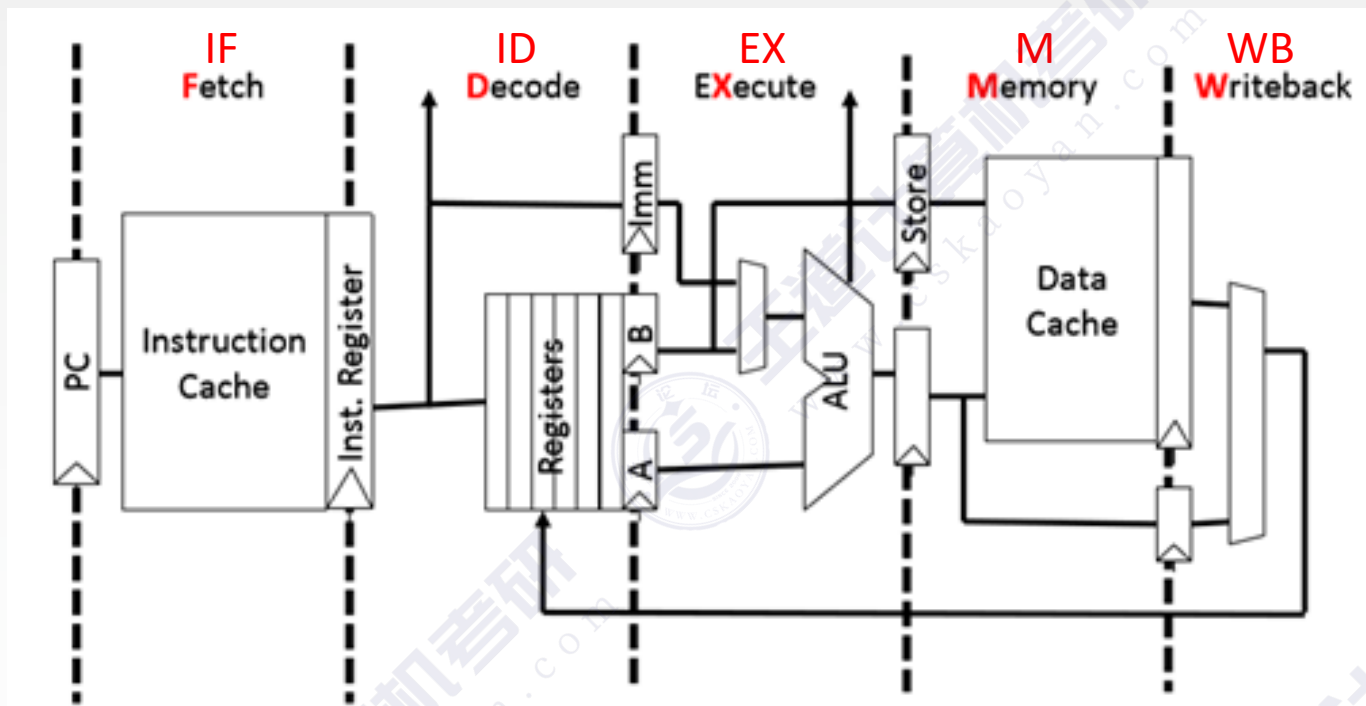
STORE Rs,mem

功能

$Rs \rightarrow (996 + (Rd))$

$Rs \rightarrow (mem)$

无条件转移指令的执行过程



转移类指令常采用相对寻址

无条件转移指令

IF: 根据PC从指令Cache取指令至IF段的锁存器

ID: 偏移量放入 Imm

EX: 将目标PC值写回PC (左图没画全)

M: 空段

WB: 空段

①IF取指 → ②ID译码&取数 → ③EX 执行 → ④M访存 → ⑤WB写回寄存器

指令的汇编格式

`jmp #偏移量`

功能

$(PC) + \text{指令字长} + (\text{偏移量} \times \text{指令字长}) \rightarrow PC$

“WrPC段”耗时比EX段更短，可安排在EX段时间内完成。WrPC段越早完成，就越能避免控制冲突。当然，也有的地方会在WB段时间内才修改PC的值

考点22

多处理器的 基本概念

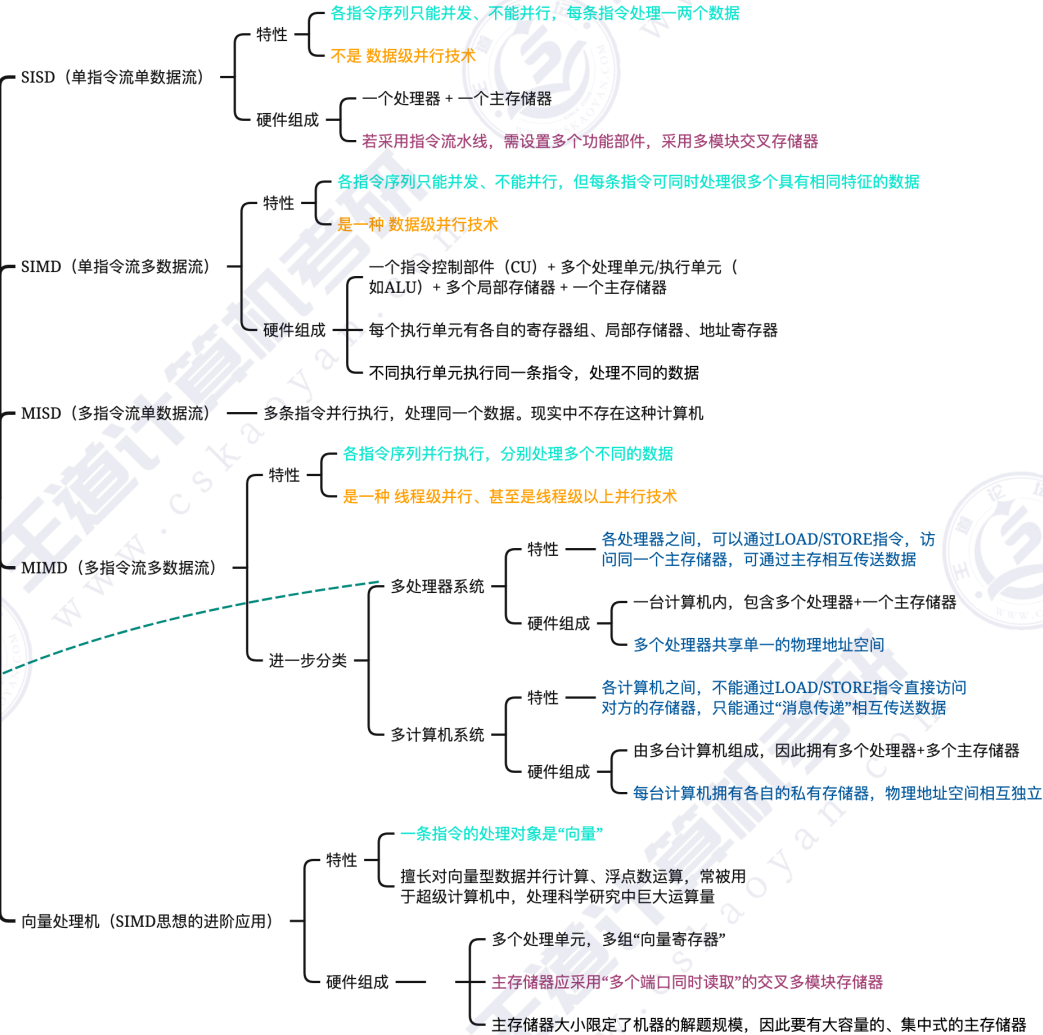
考点22：多处理器的基本概念



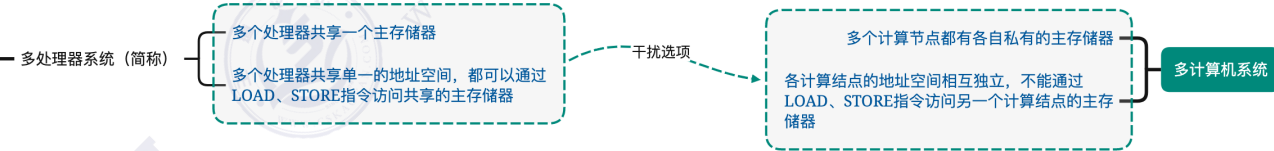
历年考频：小题×1、综合题×0

多处理器的基本概念

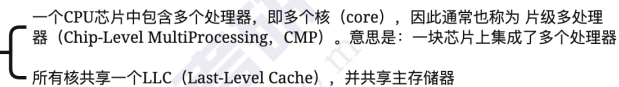
SISD、SIMD、MIMD、
向量处理器的基本概念



共享内存多处理器 (Shared Memory
multiProcessor, SMP) 的基本概念



多核处理器 (multi-core)
的基本概念



三种硬件多线程

硬件多线程			
	细粒度多线程	粗粒度多线程	同时多线程 (SMT)
指令发射	轮流发射各线程的指令 (每个时钟周期发射一个线程)	连续几个时钟周期, 都发射同一线程的指令序列, 流水线阻塞时, 切换另一个线程	一个时钟周期内, 同时发射多个线程的指令
线程切换频率	每个时钟周期切换一次线程	只有流水线阻塞时才切换一次线程	NULL
线程切换代价	低	高, 需要重载流水线	NULL
并行性	指令级并行, 线程间不并行	指令级并行, 线程间不并行	指令级并行, 线程级并行

单核CPU, 也可以实现三种硬件多线程

三种硬件多线程

时钟	CPU
i	发射线程 A 的指令j、j+1
i+1	发射线程 B 的指令k、k+1
i+2	发射线程 A 的指令j+2、j+3
i+3	发射线程 B 的指令k+2、k+3

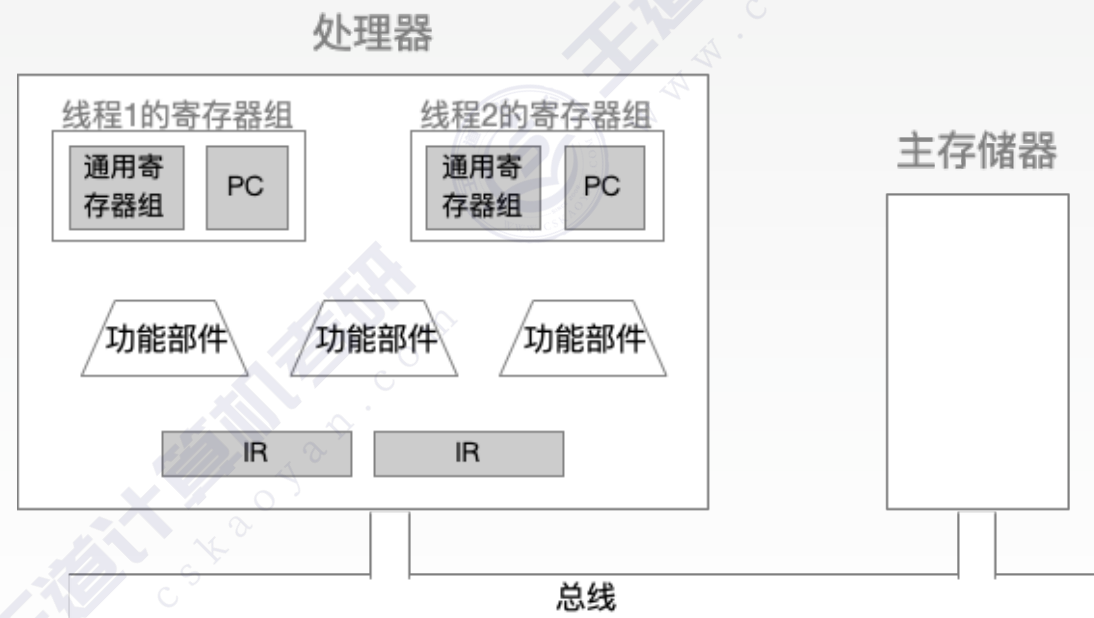
(a) 细粒度多线程示例

时钟	CPU
i	发射线程 A 的指令j、j+1
i+1	发射线程 A 的指令j+2、j+3, 发现Cache miss
i+2	线程调度, 从 A 切换到B
i+3	发射线程 B 的指令k、k+1
i+4	发射线程 B 的指令k+2、k+3

(b) 粗粒度多线程示例

时钟	CPU
i	发射线程 A 的指令j、j+1, 线程 B 的指令k、k+1
i+1	发射线程 A 的指令j+2, 线程 B 的指令k+2, 线程 C 的指令m
i+2	发射线程 A 的指令j+3, 线程 C 的指令m+1、m+2

(c) 同时多线程示例



支持硬件多线程的处理器

真题实例

22. 下列关于并行处理技术的叙述中，不正确的是

- A. 多核处理器属于 MIMD 结构
- B. 向量处理器属于 SIMD 结构
- C. 硬件多线程技术只可用于多核处理器
- D. SMP 中所有处理器共享单一物理地址空间

22. C.【解析】MIMD 结构分为多计算机系统和多处理器系统，A 正确。向量处理器是 SIMD 的变体，属于 SIMD 结构，B 正确。硬件多线程技术是在一个核中处理多个线程，可用于单核处理器，C 错误。共享内存多处理器（SMP）具有共享的单一物理地址空间，所有核都可通过存取指令来访问同一片主存地址空间，D 正确。